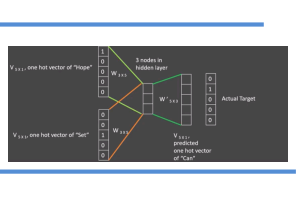
自然语言处理知识总结

自然语言处理入门

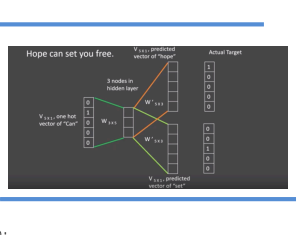
- 什么是自然语言处理
  - 计算机科学与语言学中关注于计算机与人类语言间转换的领域.
- AI几个时间点?
  - CV领域2012年分水岭
    - 2012年alxnet网络出现 用深度学习的方法解决CV问题
  - NLP领域2017年分水岭
    - 2017年之前，使用传统的机器学习技术，还有经典的循环网络来解决NLP问题
    - 2017年之后 深度学习技术来解决NLP的问题-预训练模型
- 自然语言处理应用场景
  - 语音助手
  - 机器翻译
  - 搜索引擎
  - 文本摘要
  - 文本分类

文本预处理

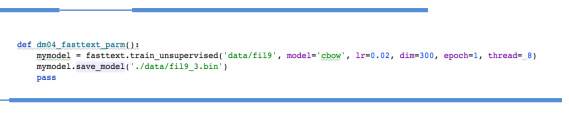
- 什么是文本预处理
  - 将文本语料进行一系列的预处理, 符合模型输入
- 文本处理的基本方法
  - 分词
    - 定义
      - 分词就是将连续的字序列按照一定的规范重新组合成词序列的过程
    - 为什么要分词?
      - 中文没有明显的单词分解符（eg英文空格是天然的分隔符）
    - 作用
      - 词作为语言语义理解的最小单元, 是人类理解文本语言的基础.
    - 分词工具
      - jieba
        - 安装
          - pip install jieba
        - 特性
          - 三种分词模式
            - 精确模式
              - 定义
                - 试图将句子最精确地切开，适合文本分析
              - API应用:
            - 全模式
              - 定义
                - 把句子中所有的可以成词的词语都扫描出来, 速度非常快，但是不能消除歧义。
              - API应用
            - 搜索引擎模式
              - 定义
                - 在精确模式的基础上，对长词再次切分，提高召回率，适合用于搜索引擎分词。
              - API应用
          - 支持繁体中文分词
          - 支持用户自定义词典
            - 作用
              - 按照词典的词语进行分词
            - API应用
              - jieba.load_userdict("./userdict.txt")
    - 应用场景
      - 文本表示
        - word2id,id2word
      - 文本相似度
        - 余弦相似度
        - 欧式距离
        - 曼哈顿距离
  - 命名实体识别
    - 定义
      - 识别出一段文本中可能存在的命名实体；实体定义:专有名词的统称 (人名 地名 机构名)
    - 作用
      - 同词汇一样, 命名实体也是人类理解文本的基础单元, 因此也是AI解决NLP领域高阶任务的重要基础环节.
    - 应用场景
      - 知识图谱
      - 文本理解
      - 舆情分析
  - 词性标注
    - 定义
      - 标注出一段文本中每个词汇的词性.
    - 作用
      - 词性标注以分词为基础, 是对文本语言的另一个角度的理解
    - 工具
      - jieba
        - API应用
      - ltp
        - API应用
    - 应用场景
      - 文本分类
      - 句法解析
- 文本张量的表示方法
  - 什么是张量
    - 将一段文本使用张量进行表示，其中一般将词汇表示成向量，称作词向量，再由各个词向量按顺序组成矩阵形成文本表示.
  - 张量的作用
    - 将文本表示成张量（矩阵）形式，能够使语言文本可以作为计算机处理程序的输入，进行接下来一系列的解析工作.
  - 实现方式
    - one-hot编码
      - 定义
        - 又称独热编码，将每个词表示成具有n个元素的向量，这个词向量中只有一个元素是1，其他元素都是0，不同词汇元素为0的位置不同，其中n的大小是整个语料中不同词汇的总数.
      - 实现方式
        - 词汇映射器Tokenizer
      - 优劣势
        - 优势
          - 操作简单，容易理解.
        - 劣势
          - 完全割裂了词与词之间的联系，而且在大语料集下，每个向量的长度过大，占据大量内存
    - Word2vec
      - 定义
        - 一种将词汇表示成向量的无监督训练方法
      - 训练模式
        - CBOW
          - 给定一段用于训练的文本语料, 再选定某段长度(窗口)作为研究对象, 使用上下文词汇预测目标词汇.
          - 网络结构



        - skipgram
          - 给定一段用于训练的文本语料, 再选定某段长度(窗口)作为研究对象, 使用目标词汇预测上下文词汇.
          - 网络结构



      - 训练和使用
        - fastText工具

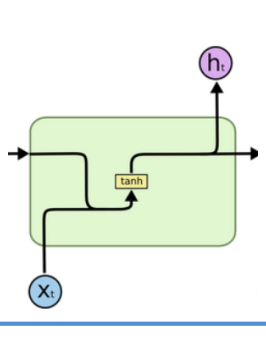


```
def dm04_fasttext_parm():
    mymodel = fasttext.train_unsupervised('data/fil9', model='cbow', lr=0.02, dim=300, epoch=1, thread= 8)
    mymodel.save_model('./data/fil9_3.bin')
    pass
```

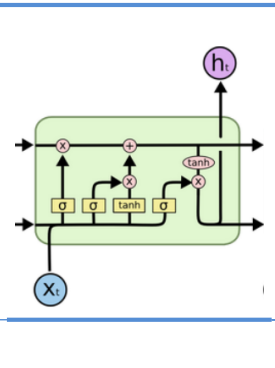
    - Word Embedding
      - 定义
        - 通过一定的方式将词汇映射到指定维度(一般是更高维度)的空间.
      - 使用
        - 在神经网络中加入的embedding层, 对整个网络进行训练的同时产生的embedding矩阵(embedding层的参数)
      - 可视化
        - tensorboard
          - # 启动tensorboard服务 tensorboard --logdir=runs --host 0.0.0.0# 通过浏览器，查看词向量可视化效果 http://127.0.0.1:6006
- 文本数据分析
  - 什么是数据分析
    - 用适当的统计分析方法对大量数据分析，以求最大化地开发数据的功能，发挥数据的作用。
  - 作用
    - 文本数据分析能够有效帮助我们理解数据语料, 快速检查出语料可能存在的问题, 并指导之后模型训练过程中一些超参数的选择.
  - 方法
    - 标签数量分布
      - 查询样本是否均衡，是否有必要进行数据增强或者数据删减
    - 句子长度分布
      - 评估语料句子长度分布范围，对于模型固定输入尺寸张量要求，选取合理范围值进行句子截断补齐
    - 词频统计与关键词词云
      - 语料质量评估，方便人工审核和修正
- 文本特征处理
  - 作用
    - 文本特征处理包括为语料添加具有普适性的文本特征, 如:n-gram特征, 以及对加入特征之后的文本语料进行必要的处理, 如: 长度规范. 这些特征处理工作能够有效的将重要的文本特征加入模型训练中, 增强模型评估指标.
  - 方法
    - 添加n-gram特征
      - 给定一段文本序列, 其中n个词或字的相邻共现特征即n-gram特征, 常用的n-gram特征是bi-gram和tri-gram特征
    - 文本长度规范
      - 定义
        - 一般模型的输入需要等尺寸大小的矩阵, 因此在进入模型前需要对每条文本数值映射后的长度进行规范，对超长文本进行截断, 对不足文本进行补齐(一般使用数字0)
      - 实现
- 文本数据增强
  - 定义
    - 主要通过人工构建数据，增加训练集的大小使模型达到更好的泛化特性
  - 作用
    - 对于不平衡数据集很有用，可以最大程度地减少标注工作
  - 方法
    - 回忆数据增强法
      - 定义
        - 基于google翻译接口, 将文本数据翻译成另外一种语言，之后再翻译回原语言,得到新语料, 完成对原数据集数据增强.
      - 优势
        - 操作简便, 获得新语料质量高
      - 问题及解决方法
        - 在短文本回译过程中, 新语料与原语料可能存在很高的重复率, 并不能有效增大样本的特征空间.
        - 进行连续的多语言翻译
      - 实现方式
        - translator = google_translator() translator.translate(text, lang_src='zh-cn', lang_tgt='ko')

RNN及其变体

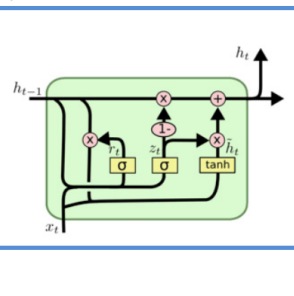
- 什么是RNN
  - 循环神经网络：它一般以序列数据为输入, 通过网络内部的结构设计有效捕捉序列之间的关系特征, 一般也是以序列形式进行输出.
- RNN模型作用及类别
  - 作用
    - RNN结构能够很好利用序列之间的关系, 因此针对自然界具有连续性的输入序列, 如人类的语言, 语音等进行很好的处理, 广泛应用于NLP领域的各项任务, 如文本分类, 情感分析, 意图识别, 机器翻译等.
  - RNN模型类别
    - 输入与输出结构分类
      - N vs N
        - 特点
          - 输入与输出等长
        - 应用范围
          - 生成等长度的合辙诗句
      - N vs 1
        - 特点
          - 输入是一个序列，而要求输出是一个单独的值
        - 应用范围
          - 文本分类
      - 1 vs N
        - 特点
          - 输入不是序列而输出为序列
        - 应用范围
          - 图片生成文字任务
      - N vs M
        - 特点
          - 输入输出均不受限制
        - 应用范围
          - 机器翻译，阅读理解，文本摘要等
    - 内部构造分类
      - 传统RNN
      - LSTM
      - GRU
- 传统RNN
  - 模型结构



    - 输入：h(t-1)以及x(t)
    - 输出：output以及h(t)
  - 模型构建
  - 优点
    - 由于内部结构简单,参数量少， 对计算资源要求低,
  - 缺点
    - 传统RNN在解决长序列之间的关联时,表现很差, 原因是在进行反向传播的时候, 过长的序列导致梯度的计算异常, 发生梯度消失或爆炸.
- LSTM
  - 模型结构



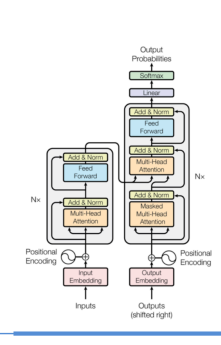
    - 输入门
    - 细胞状态
    - 遗忘门
    - 输出门
  - 模型构建
  - 优点
    - 传统RNN的变体, 与经典RNN相比能够有效捕捉长序列之间的语义关联, 缓解梯度消失或爆炸现象
  - 缺点
    - 内部结构相对较复杂，训练效率在同等算力下较传统RNN低很多
  - BI-LSTM
    - Bi-LSTM即双向LSTM, 它没有改变LSTM本身任何的内部结构, 只是将LSTM应用两次且方向不同, 再将两次得到的LSTM结果进行拼接作为最终输出.
- GRU
  - 模型结构



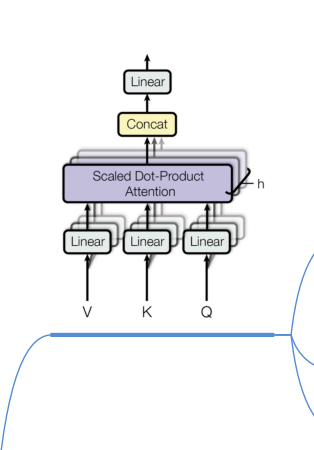
    - 重置门
    - 更新门
  - 模型构建
  - 优点
    - GRU和LSTM作用相同, 在捕捉长序列语义关联时, 能有效抑制梯度消失或爆炸, 效果都优于传统RNN且计算复杂度相比LSTM要小.
  - 缺点
    - GRU仍然不能完全解决梯度消失问题, 不可并行计算, 这在数据量和模型体量逐步增大的未来, 是RNN发展的关键瓶颈.
  - BI-GRU
    - Bi-GRU即双向GRU 它没有改变GRU本身任何的内部结构, 只是将GRU应用两次且方向不同, 再将两次得到的GRU结果进行拼接作为最终输出.
- 案例1：人名分类器 ㊴
- 注意力机制
  - 定义
    - 在认知科学中，由于信息处理的瓶颈，人类会选择性地关注所有信息的一部分，同时忽略其他可见的信息。
  - 注意力计算
    - 计算说明
      - 指定三个输入：Q、K、V，然后通过计算公式得到计算结果
    - 计算规则
      - 将Q，K进行纵轴拼接, 做一次线性变化, 再使用softmax处理获得结果最后与V做张量乘法
      - 将Q，K进行纵轴拼接, 做一次线性变化后再使用tanh函数激活, 然后再进行内部求和, 最后使用softmax处理获得结果再与V做张量乘法.
      - 将Q与K的转置做点积运算, 然后除以一个缩放系数, 再使用softmax处理获得结果最后与V做张量乘法.
  - 深度神经网络注意力机制
    - 定义
      - 注意力机制是注意力计算规则能够应用的深度学习网络的载体, 同时包括一些必要的全连接层以及相关张量处理, 使其与应用网络融为一体
    - 作用
      - 编码器端
        - 主要解决表征问题, 相当于特征提取过程, 得到输入的注意力表示. 一般使用自注意力(self-attention).
      - 解码器端
        - 能够根据模型目标有效的聚焦编码器的输出结果, 当其作为解码器的输入时提升效果. 改善以往编码器输出是单一定长张量, 无法存储过多信息的情况
    - 实现步骤
      - 第一步：根据注意力计算规则, 对Q，K，V进行相应的计算.
      - 第二步: 根据第一步采用的计算方法, 如果是拼接方法，则需要将Q与第一步的计算结果再进行拼接, 如果是转置点积, 一般是自注意力, Q与V相同, 则不需要进行与Q的拼接.
      - 第三步: 最后为了使整个attention机制按照指定尺寸输出, 使用线性层作用在第二步的结果上做一个线性变换, 得到最终对Q的注意力表示.
- 案例2：seq2seq英译法 (58)

Transformer

- 什么是transformer
  - 一种利用注意力机制来提高模型训练速度的模型
- transformer的作用
  - 基于seq2seq架构的transformer模型可以完成NLP领域研究的典型任务, 如机器翻译, 文本生成等. 同时又可以构建预训练语言模型，用于不同任务的迁移学习.
- transformer模型架构



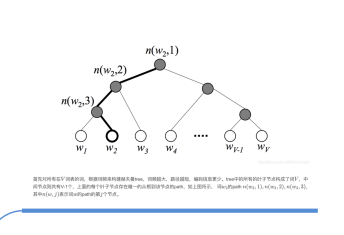
  - 输入部分
    - word Embedding
    - Positional Encoding
      - 作用
        - 将词汇位置不同可能会产生不同语义的信息加入到词嵌入张量中, 以弥补位置信息的缺失.
      - 实现方式
        - 三角函数
  - 输出部分
    - 线性层
    - softmax层
  - 编码器部分
    - N个编码器层堆叠
    - 每个编码器层由两个子层连接结构组成
    - 第一个子层连接结构包括一个多头自注意力层和规范化层以及一个残差连接
      - 多头自注意力机制



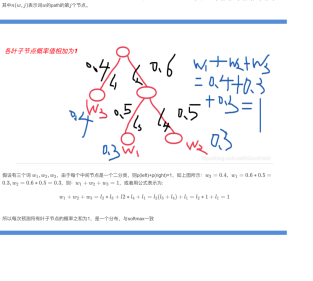
        - 概念
          - 基于相同的注意力机制学习到不同的行为，然后将不同的行为作为知识组合起来，捕获序列内各种范围的依赖关系
        - 作用
          - 每个注意力机制去优化每个词汇的不同特征部分，从而均衡同一种注意力机制可能产生的偏差，让词义拥有来自更多元的表达，实验表明可以从而提升模型效果.
        - 形式
          - Q=K=V，对三个变量分别进行线性变换
        - 掩码张量应用
          - 定义
            - 掩代表遮掩，码就是我们张量中的数值，它的尺寸不定，里面一般只有1和0的元素，代表位置被遮掩或者不被遮掩，至于是0位置被遮掩还是1位置被遮掩可以自定义，因此它的作用就是让另外一个张量中的一些数值被遮掩，也可以说被替换, 它的表现形式是一个张量.
          - 作用
            - 未来的信息可能被提前利用. 所以，我们会进行遮掩
          - 使用
            - scores.masked_fill(mask == 0, -1e9)
      - 规范化层
        - 随着网络层数的增加，通过多层的计算后参数可能开始出现过大或过小的情况，这样可能会导致学习过程出现异常，模型可能收敛非常的慢. 因此都会在一定层数后接规范化层进行数值的规范化，使其特征数值在合理范围内.
      - 残差连接
        - 为了将信息传递的更深, 增强模型的拟合能力.
    - 第二个子层连接结构包括一个前馈全连接层和规范化层以及一个残差连接
      - 前馈全连接层
        - 定义
          - 具有两层线性层的全连接网络.
        - 作用
          - 考虑注意力机制可能对复杂过程的拟合程度不够, 通过增加两层网络来增强模型的能力.
  - 解码器部分
    - N个解码器层堆叠
    - 每个解码器层由三个子层连接结构组成
    - 第一个子层连接结构包括一个多头自注意力层和规范化层以及一个残差连接
    - 第二个子层连接结构包括一个多头注意力层和规范化层以及一个残差连接
      - 注意
        - 多头注意力对象，这里Q!=K=V
    - 第三个子层连接结构包括一个前馈全连接层和规范化层以及一个残差连接
- transformer优势
  - 可以实现并行计算
  - 采用Muti-head Attention结构和计算机制，拥有更强大的特征处理能力

迁移学习

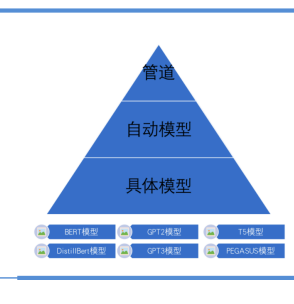
- fasttext工具介绍
  - 定义
    - 一种NLP工程领域常用的工具包
  - 作用
    - 进行文本分类
    - 训练词向量
  - 优点
    - 速度快
    - 准cpu下, 10亿词汇10分钟
  - 原因
    - 模型简单
    - 使用层次softmax结构，来提升超多类别下的模型性能
      - 为什么层次softmax要比普通softmax要快
        - 根据词频，把所有单词都挂在霍夫曼树的叶子节点上
        - 每个单词到根节点的路径path唯一
        - 出现频率越大越接近根节点，也就是路径越短



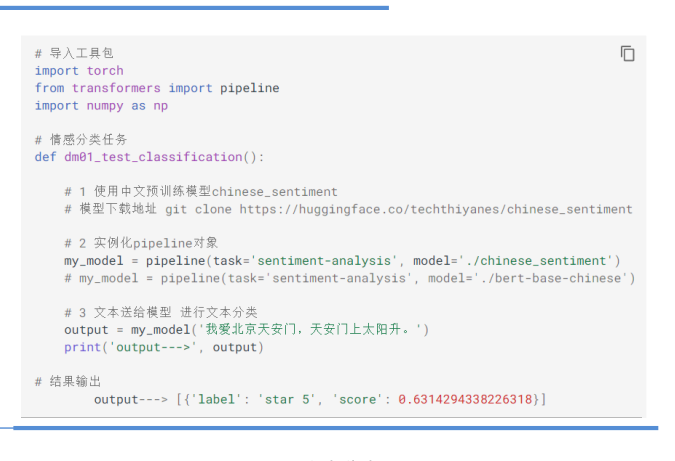
        - 求每个单词的概率分布转化为求单词路径的概率乘积



    - 添加n-gram特征已弥补模型缺陷提升精度
  - 安装
    - pip install fasttext
- fasttext文本分类
  - 什么是文本分类
    - 将文档分配给一个或多个类别
  - 文本分类种类
    - 二分类
      - 文本被分类两个类别中, 往往这两个类别是对立面, 比如: 判断一句评论是好评还是差评.
    - 单标签多分类
      - 文本被分入到多个类别中, 且每条文本只能属于某一个类别
    - 多标签多分类
      - 文本被分到多个类别中, 但每条文本可以属于多个类别
  - 文本分类过程
    - 第一步：获取数据
    - 第二步：训练集与验证集的划分
    - 第三步：训练模型
    - 第四步：使用模型进行预测并评估
    - 第五步：模型调优
      - 模型优化方法
        - 原始数据处理，脏数据处理
        - 增加训练轮数
        - 调整学习率
        - 增加n-gram特征
        - 修改损失计算方式
          - model4 = fasttext.train_supervised(input="cooking.train", lr=1.0, epoch=30, wordNgrams=2, **loss='hs'**)
          - 可提高模型训练速度, 但会损失一部分精度 要看工程的需要
        - 自动超参数调优
          - model5 = fasttext.train_supervised(input='cooking.train', autotuneValidationFile='cooking.valid', autotuneDuration=600)
        - 实际生产中多标签多分类问题的损失计算方式
          - model6 = fasttext.train_supervised(input="cooking.train", lr=0.2, epoch=30, wordNgrams=2, **loss='ova'**)
          - 准换为多个二分类
    - 第六步：模型的保存和加载
      - modle.save_model(path)；
      - fasttext.load_model(path)
- 训练词向量
  - 作用
    - 用向量表示文本中的词汇
  - 实现步骤
    - 第一步：获取数据
    - 第二步：训练词向量
    - 第三步：模型超参数设定
    - 第四步：模型效果检验
    - 第五步：模型的保存和加载
- 词向量迁移
  - 什么是词向量迁移
    - 使用在大型语料库上已经进行训练完成的词向量模型.
  - fasttext可以提供的可迁移的词向量
    - fasttext提供了157种语言的在CommonCrawl和Wikipedia语料上进行训练的可迁移词向量模型, 它们采用CBOW模式进行训练, 词向量维度为300维.
    - fasttext提供了294种语言的在Wikipedia语料上进行训练的可迁移词向量模型, 它们采用skipgram模式进行训练, 词向量维度同样是300维.
  - 使用fasttext进行词向量迁移
    - 第一步: 下载词向量模型压缩的bin.gz文件
    - 第二步: 解压bin.gz文件到bin文件
    - 第三步: 加载bin文件获取词向量
      - api
        - 加载词向量模型model = fasttext.load_model("cc.zh.300.bin")
        - 获取词向量 model.get_word_vector("the")
    - 第四步: 利用邻近词进行效果检验
      - api
        - 获取临近词 model.get_nearest_neighbors("音乐")
- 迁移学习
  - 预训练模型
    - 定义 ①
    - 预训练语言模型分类 ①
  - 微调
    - 根据给定的预训练模型，改变它的部分参数或者为其新增部分输出结构后，通过在小部分数据集上训练，来使整个模型更好的适应特定任务。
  - 迁移方式
    - 直接使用预训练模型，开箱即用的效果
    - 微调方式，通过训练更新小部分参数以此适应不同任务
- NLP中的标准数据集
  - 作用 ①
  - 数据集类型
    - CoLA数据集
      - 释义
        - 纽约大学发布的有关语法的数据集
      - 任务本质
        - 是对一个给定句子，判定其是否语法正确的单个句子的**二分类任务**.
    - SST-2数据集
      - 释义
        - 单句子分类任务，包含电影评论中的句子和它们情感的人类注释.
      - 任务本质
        - 句子级别的**二分类任务**
    - MRPC数据集
      - 释义
        - 相似性和释义任务，是从在线新闻源中自动抽取句子对语料库，并人工注释句子对中的句子是否在语义上等效.
      - 任务本质
        - 句子级别的**二分类任务*
    - STS-B数据集
      - 释义
        - 语义文本相似性基准测试
      - 任务本质
        - 回归任务/句子对的文本五分类任务
    - QQP数据集
      - 释义
        - 相似性和释义任务，是社区问答网站Quora中问题对的集合
      - 任务本质
        - 句子对的**二分类任务**
    - (MNLI/SNLI)数据集
      - 释义
        - 多类型自然语言推理数据库
      - 任务本质
        - 句子对的**三分类任务**
    - (QNLI/RTE/WNLI)数据集
      - 释义
        - QNLI，问答自然语言推断
        - RTE，识别文本蕴含数据集
        - WNLI，Winograd自然语言推断
      - 任务本质
        - 均属于二分类任务
- NLP中常用预训练模型
  - BERT
    - 介绍
      - 2018年10月由Google AI研究院提出的一种预训练模型(自编码)
      - 11种不同NLP测试中创出SOTA表现
    - 架构
      - Embedding模块
        - Token Embddings
          - 词嵌入张量
        - Segment Embeddings
          - 句子分段嵌入张量
        - Position Embeddings
          - 位置编码张量：通过学习而来，非三角函数定义
      - Transformer模块
        - 双向Transformer模块
          - BERT中只使用了经典Transformer架构中的Encoder部分, 完全舍弃了Decoder部分. 而两大预训练任务也集中体现在训练Transformer模块中.
      - 预微调模块
        - 经过中间层Transformer的处理后, BERT的最后一层根据任务的不同需求而做不同的调整即可.
    - 预训练任务
      - MLM任务(Masked Language Model)
        - 原始文本中随机抽取15%的token参与任务
          - 80%概率，用【MASK】替换
          - 在10%概率下, 用一个随机的单词替换该token.
          - 在10%概率下, 保持该token不变
      - NSP任务(Next Sentence Prediction)
        - 采用的方式是输入句子对(A, B), 模型预测句子B是不是句子A的真实的下一句话.
          - 其中50%的B是原始文本中真实跟随A的下一句话.(标记为IsNext, 代表正样本)
          - 其中50%的B是原始文本中随机抽取的一句话.(标记为NotNext, 代表负样本)
    - 类型
      - 常用bert_base_chinese
        - 编码器具有12个隐层, 输出768维张量, 12个自注意力头, 共110M参数量
  - GPT
    - 介绍
      - GPT是OpenAI公司提出的一种预训练语言模型（自回归）
      - 本质上来说, GPT是一个单向语言模型.
    - 模型架构
      - GPT采用了Transformer架构中的解码器模块.
      - GPT在使用解码器模块时做了一定的改造, 将传统的3层Decoder Block变成了2层Block, 删除了encoder-decoder attention子层, 只保留Masked Multi-Head Attention子层和Feed Forward子层
      - GPT的解码器总共是由12个改造后的Decoder Block组成的.
    - 预训练任务
      - 第一阶段: 无监督的预训练语言模型. 只利用单词前面的信息来预测当前单词
      - 第二阶段: 有监督的下游任务fine-tunning.
    - 类型
      - gpt
        - 编码器具有12个隐层, 输出768维张量, 12个自注意力头, 共110M参数量
  - GPT-2
    - 介绍
      - GPT2是在GPT基础上发展处的更强大的语言预训练模型.
    - 类型
      - gpt2
        - 编码器具有12个隐层, 输出768维张量, 12个自注意力头, 共117M参数量
      - gpt2-xl
        - 编码器具有48个隐层, 输出1600维张量, 25个自注意力头, 共1558M参数量
  - Tansformer-XL
    - transfo-xl-wt103
      - 编码器具有18个隐层, 输出1024维张量, 16个自注意力头, 共257M参数量
  - XLNet及变体
    - xlnet-base-cased
      - 编码器具有12个隐层, 输出768维张量, 12个自注意力头, 共110M参数量
    - xlnet-large-cased
      - 编码器具有24个隐层, 输出1024维张量, 16个自注意力头
  - XLM
    - xlm-mlm-en-2048
      - 编码器具有12个隐层, 输出2048维张量, 16个自注意力头
  - RoBERTa
    - roberta-base
      - 编码器具有12个隐层, 输出768维张量, 12个自注意力头, 共125M参数量
    - roberta-large
      - 编码器具有24个隐层, 输出1024维张量, 16个自注意力头
  - ALBERT
    - albert-base-v1
      - 编码器具有12个隐层, 输出768维张量, 12个自注意力头, 共125M参数量
    - albert-base-v2
      - 子编码器具有12个隐层, 输出768维张量, 12个自注意力头, 共125M参数量
- Tansformers库的使用
  - 介绍
    - 基于 transformer 模型结构的预训练语言库(开源)
    - Transformers 提供了NLP领域大量预训练语言模型结构的模型和调用框架
  - 应用结构

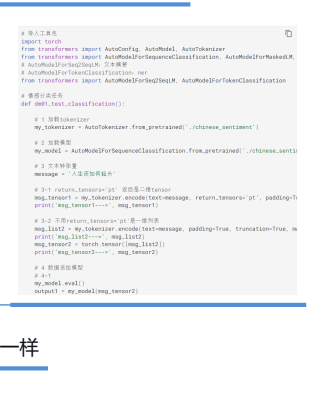


    - 管道（Pipline）方式
      - 高度集成的极简使用方式，只需要几行代码即可实现一个NLP任务
    - 具体模型（SpecificModel）方式
      - 在使用时，需要明确指定具体的模型
    - 自动模型（AutoMode）方式
      - 可载入并使用BERTology系列模型。
  - 使用管道方式完成多种NLP任务
    - 如何使用
      - 基本步骤
        - 第一步：导入工具包：from transformers import pipline
        - 第二步：直接根据NLP任务对pipline类进行实例化， 便可以得到能够使用的模型对象，这里模型可以选择
        - 第三步：将文本输入模型对象，进行具体NLP任务处理
      - API调用举例:



```
# 导入工具包
import torch
from transformers import pipeline
import numpy as np

# 情感分类任务
def dm01_test_classification():

    # 1 使用中文预训练模型chinese_sentiment
    # 模型下载地址 git clone https://huggingface.co/techthiyanes/chinese_sentiment

    # 2 实例化pipeline对象
    my_model = pipeline(task='sentiment-analysis', model='./chinese_sentiment')
    # my_model = pipeline(task='sentiment-analysis', model='./bert-base-chinese')

    # 3 文本送给模型 进行文本分类
    output = my_model('我爱北京天安门，天安门上太阳升。')
    print('output--->', output)

# 结果输出
        output---> [{'label': 'star 5', 'score': 0.6314294338226318}]
```

    - 可实现任务
      - 带头任务：根据任务类型，输出相应类型结果
        - 文本分类
        - 完形填空
        - 阅读理解
        - 文本摘要
        - NER
      - 不带头任务：文本张量的表示
        - 特征提取任务
  - 使用自动模型方式完NLP任务
    - 如何使用
      - 基本步骤
        - 第一步：导入工具包：from transformers import *
        - 第二步：加载对应具体任务的tokenizer
        - 第三步：加载对应具体任务的模型
        - 第五部：将文本输入模型对象，进行具体NLP任务处理
        - 第四步：文本张量转化
      - API调用举例



```
# 导入工具包
import torch
from transformers import AutoConfig, AutoModel, AutoTokenizer
from transformers import AutoModelForSequenceClassification, AutoModelForMaskedLM,
# AutoModelForSeq2SeqLM: 文本摘要
# AutoModelForTokenClassification: ner
from transformers import AutoModelForSeq2SeqLM, AutoModelForTokenClassification

# 情感分类任务
def dm01_test_classification():

    # 1 加载tokenizer
    my_tokenizer = AutoTokenizer.from_pretrained('./chinese_sentiment')

    # 2 加载模型
    my_model = AutoModelForSequenceClassification.from_pretrained('./chinese_senti

    # 3 文本转张量
    message = '人生该如何起头'

    # 3-1 return_tensors='pt' 返回是二维tensor
    msg_tensor1 = my_tokenizer.encode(text=message, return_tensors='pt', padding=T
    print('msg_tensor1--->', msg_tensor1)

    # 3-2 不用return_tensors='pt' 是一维列表
    msg_list2 = my_tokenizer.encode(text=message, padding=True, truncation=True, m
    print('msg_list2--->', msg_list2)
    msg_tensor2 = torch.tensor([msg_list2])
    print('msg_tensor2--->', msg_tensor2)

    # 4 数据送给模型
    # 4-1
    my_model.eval()
    output1 = my_model(msg_tensor2)
```

    - 可实现任务
      - 同pipline实现任务一样
  - 使用具体模型方式完成NLP任务
    - 如何使用
      - 步骤和自动模型一样
      - API调用举例 ①
    - 可实现任务
      - 同上
- 迁移学习任务实践 ㉓

Presented with xmind